

Lab 1 — AI Use and Reflection

LLM/agent used: ChatGPT

#	Prompt (summarised)	What I did with the result
1	Asked how to implement <code>checkSystem()</code> in <code>api.ts</code> , including the <code>/api/health</code> and <code>/api/categories</code> fetch calls, error handling, and returning <code>{ online: true, categories }.</code>	Implemented both fetch calls, added <code>await</code> , threw errors when either request failed, and returned the expected object.
2	Asked how to implement the React UI logic in <code>App.tsx</code> , including the loading, success, and error states, calling <code>checkSystem()</code> , storing categories, and showing a useful error message.	Added the async <code>handleCheck</code> function, updated the UI state transitions, stored the fetched categories, and rendered the Online/Offline messages and category list.
3	Asked to fill the AI use documentation for the lab, including summarising prompts, mentioning the relevant files (<code>api.ts</code> and <code>App.tsx</code>), and formatting the reflection section.	Used the suggested format to complete the AI use table and reflection with concise summaries and file-specific context.
4	Asked how to implement the Prisma seed script using <code>prisma.category.upsert()</code> so the four categories are created without duplicates when the seed is run multiple times.	Implemented an idempotent seed script by looping through the four category names and calling <code>prisma.category.upsert()</code> with <code>where: { name }</code> , <code>update: {}</code> , and <code>create: { name }</code> , then logged a success message.
5	Asked how to create a PostgreSQL database matching the <code>DATABASE_URL</code> using both a single Docker command and a separate Docker Compose file, including the command to run it.	Created a Docker command using PostgreSQL 16 Alpine with the <code>toktickit</code> user, password, database, and port 5432. Also created a separate Docker Compose configuration with the same settings and provided the command to start the database.

LLM/agent used: Claude Sonnet 5

#	Prompt (summarised)	What I did with the result
1	Asked for a simple <code>categories.test.ts</code> , asserting <code>GET /api/categories</code> , with context about the <code>categories.test.ts</code> header and category order.	Implemented the test to return HTTP 200 and verify the four seeded category names and IDs in order.

LLM/agent used: VScode Copilot Extension

#	Prompt (summarised)	What I did with the result
1	Asked how to implement <code>GET /api/categories</code> with context of <code>app.ts</code> .	Added the Prisma query, ordered results by <code>id</code> , and returned a safe error response.
2	Asked to update <code>App.test.tsx</code> tests for Issue 4 with the required categories, without changing the header or adding other libraries, using context from <code>App.tsx</code> and <code>api.ts</code> .	Added two tests and mocked <code>checkSystem</code> to verify the success and error UI states.

Reflection

Giving the file name, file information and the required behavior made the prompts much more accurate because the AI could generate code that matched the project structure. Also restriting ai to follow the format of the original file is nesscessary in some work sice somtimes is just swithc to a diffrent library espeacly on the test file.